Version of Record: https://www.sciencedirect.com/science/article/pii/S0010448516301117
Manuscript_cc6ca2acb4ff68a0aef947ac688dbca4

A Template-Based Approach for Parallel Hexahedral Two-Refinement

Steven J. Owen[a,*], Ryan M. Shih[b], Corey D. Ernst[1]

[a] *Sandia National Laboratory, Albuquerque, New Mexico, USA.*
[b] *University of California, Berkeley, California, USA.*
[c] *Elemental Technologies, American Fork, Utah, USA.*

Abstract

We provide a template-based approach for generating locally refined all-hex meshes. We focus specifically on refinement of initially structured grids utilizing a 2-refinement approach where uniformly refined hexes are subdivided into eight child elements. The refinement algorithm consists of identifying marked nodes that are used as the basis for a set of four simple refinement templates. The target application for 2-refinement is a parallel grid-based all-hex meshing tool for high performance computing in a distributed environment. The result is a parallel consistent locally refined mesh requiring minimal communication and where minimum mesh quality is greater than scaled Jacobian 0.3 prior to smoothing.

Keywords: hexahedron; mesh refinement; 2-refinement; adaptivity; parallel;

1. Introduction

Massively parallel platforms, such as those deployed at the U.S. Department of Energy Laboratories, have enabled computational simulation of enormous complexity. For applications requiring hexahedral elements, traditional methods of mesh generation can require significant user interaction which will not easily

*Sandia National Laboratories is a multi-program laboratory managed and operated by Sandia Corporation, a wholly owned subsidiary of Lockheed Martin Corporation, for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-AC04-94AL85000

© 2016 published by Elsevier. This manuscript is made available under the Elsevier user license
https://www.elsevier.com/open-access/userlicense/1.0/

scale for these problems. Fully automatic, scalable and embedded meshing methods are an increasingly important requirement for these next-generation computing platforms. Mesh generation based on an overlay grid procedure is an ideal candidate for high performance computing, however to be effective it must provide for geometry-sensitive mesh size adaptation.

Overlay grid procedures for generating all-hex meshes [1][2][3] usually rely on some form of refinement strategy to capture small features. Most of these methods begin with a regular three-dimensional Cartesian grid that is adaptively refined based on various geometric criteria to form an octree subdivided mesh. A Boundary representation (B-rep) of the geometry of interest is then superimposed on the octree mesh, where nodes are snapped to the geometry and elements falling outside of the B-Rep are discarded.

In order to maintain continuity between refined and unrefined elements in the mesh, transition patterns are normally imposed. These patterns can utilize either a 3-refinement or a 2-refinement methodology. For 3-refinement, each edge of the uniformly refined grid is divided into three segments. On a three-dimensional hex element, this results in a 3x3x3 subdivision or 27 elements. 2-refinement, on the other hand, will split each edge in two resulting in a 2x2x2 subdivision with 8 elements.

Most refinement operations can be thought of as introducing a pillow layer of hexes surrounding a column of hexes. Figure 1 illustrates a 2D 3-refined mesh where the green elements were initially marked for uniform refinement. An example continuous pillow layer of quads (shown in red) is shown wrapping a single column of quads, noting that the same pattern is repeated throughout the mesh. Figure 2 illustrates a 2-refined mesh with a similar pillow layer surrounding two columns of quads. In 3-refinement, We note that the pillow layer can be accomplished within a single quad layer, whereas 2-refinement requires at least a pair of quad columns to accomplish the pillow. This problem extends to 3D (shown in figure 3) where sheets of hexes must be introduced to accomplish the refinement. For 3-refinement, since the pillowed sheet of hexes can be effected within a single column of hexes, each refinement transition can

be performed independently and within a single element, making its implementation relatively straight-forward. In contrast, 2-refinement must determine a consistent pairing of hex layers to effect the refinement transitions, making its implementation more challenging.

The 3-refinement strategy can produce undesirable high mesh size gradients in transition regions. In spite of this, it remains the most popular form of refinement because of its ease of implementation. The 3-refinement pattern, illustrated in figure 1, has well-defined templates for transition elements that can be introduced based on a set of marked nodes. First introduced by Schneiders [4] the number and pattern of marked nodes on an element define the precise subdivision template to be used. This deterministic template-based approach to refinement, is relatively well-understood and easy to implement. Although beneficial for implementation, the change in element size and resulting mesh quality in the transition regions for 3-refinement can be problematic.

In contrast, 2-refinement, illustrated in figure 2, by most measures is a more desirable approach because of the smoother size transitions that can be effected. Its implementation, however, can be more difficult, particularly for parallel distributed domains. Complications in 2-refinement can arise when the pairing patterns to form transition elements from nearby refinement zones intersect with each other, or when the pairing of element layers must extend across processor boundaries. Several methods have been proposed which appear to present good results for serial applications, however implementation details for some of these methods are sparse and their application to distributed memory parallel is not addressed.

We address the need for hexahedral mesh refinement for distributed memory parallel environments. A refinement strategy that uses a deterministic algorithm that yields the same results regardless of the domain decomposition strategy is desirable. Because templates can provide local criteria for subdivision, they provide an attractive solution for parallel applications where very little interprocessor communication is necessary.

In this work we introduce a new deterministic 2-refinement strategy based

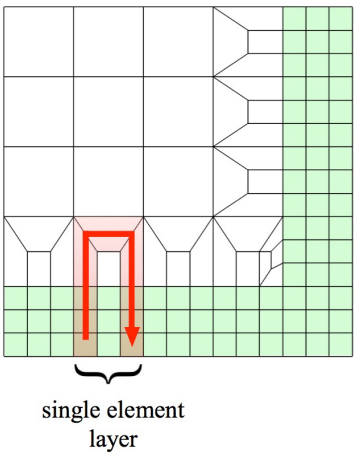


Figure 1: Example 2D 3-refinement showing pillow loops accomplished within a single layer of elements.

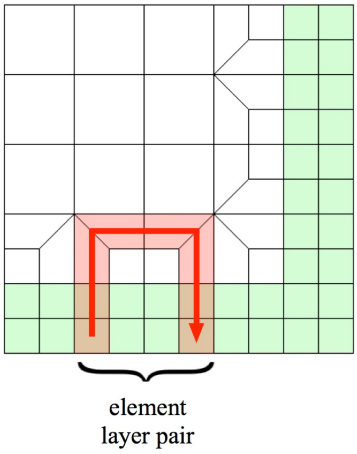


Figure 2: Example 2D 2-refinement showing pillow loops requiring at least a pair of adjacent element layers.

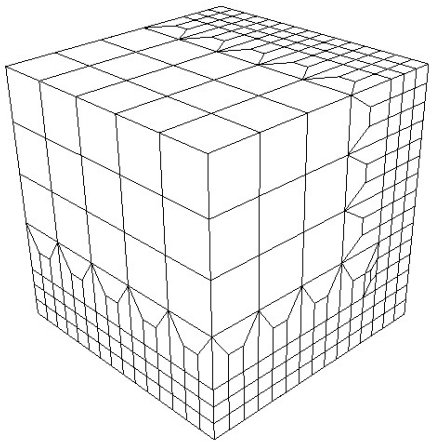


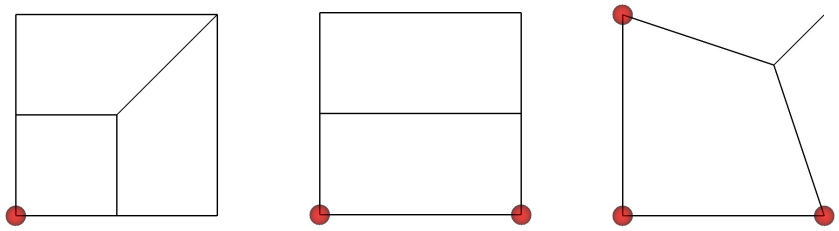
Figure 3: Example 3D mesh using a 3-refinement procedure

on templates and marked nodes. We limit our scope to structured grids since they are used as the basis for our target overlay grid application [5]. We propose a method that provides the following characteristics:

1. One-to-eight uniform refinement
2. Controlled mesh size gradients
3. Minimum mesh quality within transition elements of 0.3 scaled Jacobian
4. Parallel-consistent (Same result regardless of decomposition)
5. Scalable to massively parallel environments
6. Multiple levels of refinement

2. Previous Work

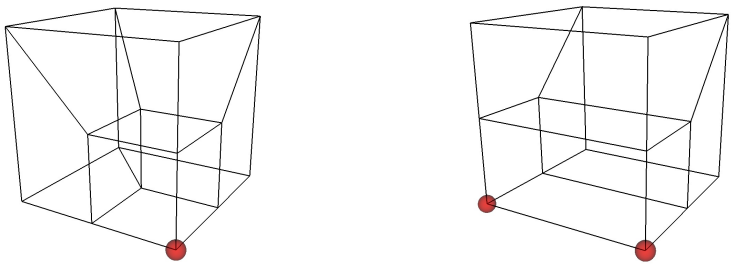
Most localized hex refinement strategies in the literature have largely been based on Schneiders' initial work on Octree meshing [4]. A detailed set of 3-refinement templates were developed and used to locally capture feature sizes. Schneiders later expands these procedures to include 2-refinement [6], introducing templates similar to those shown in figures 4 and 5. In this work, the concepts of directionally refined parallel layers is introduced, as well as a node marking strategy based on alternating nodes surrounding the refinement region to enforce pairing. We note that because of the selected overlapping strategy of the intersecting pillowing [7] operations, that transitions can quickly become complex, especially for concave regions reducing resulting mesh quality.



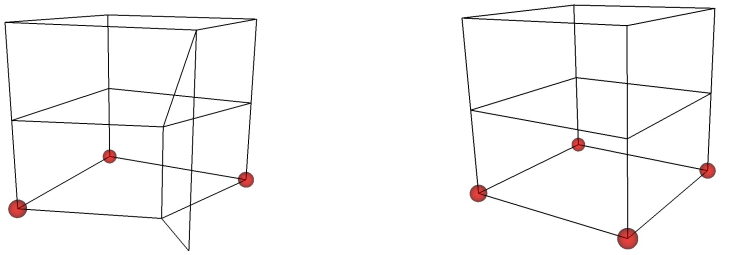
(a) *1-template* (b) *2-template* (c) *3-template*

Figure 4: Complete set of 2D 2-refinement templates. Templates are identified based on number of marked nodes on a quad or hex face. Note that for 2D, only the 1-template and 2-template are required. The 2D *3-template* is used only in the context of 3-refinement for refining a hex face of the 3D *3-template* (Fig. 5c).

Harris [8] et. al., and later expanded by Edgel [9] and Malone [10] also provide background and implementation of a directional parallel pillowing strategy for 2-refinement. These works do not address the use of templates, instead use pillow operations on arbitrarily selected adjacent hex layers based on expansion from an initial uniform refinement zone. Mesh quality in transition zones as well as generality and extension for concave regions also was problematic in these works. In contrast, in this work, we extend Harris' parallel pillowing strategy to use a global deterministic template-based approach that is applicable for any shaped region while maintaining mesh quality in excess of 0.3 scaled Jacobian



(a) *1-template* (b) *2-template*



(c) *3-template* (d) *4-template*

Figure 5: Complete set of 3D 2-refinement templates. Templates are identified based on the number of marked nodes on a hex

in transition regions.

Other 2-refinement strategies include Ebedia et. al. [11], Marechal [3] and Zhang et. al. [12]. While each of these methods offers unique benefits, they each begin with a balanced octree decomposition of an initial uniform grid. They later define column groupings of four elements extended from the octree surrounding a common edge. Transitions are generated to cap these column groupings using a set of four *1-templates* as shown in figure 5a. To handle concave regions, Ebedia [11] and Zhang [12] utilize a buffer pillow layer surrounding the uniformly refined hexes through which the transition elements are defined. Although specific metrics are not reported for elements in the transition region, it appears the elements formed in the pillowed transition layers would likely be poor.

Ebeida [11] and Zhang [12] also demonstrate several cases for reconciling non-manifold conditions, a consequence of transition regions from nearby uniform refinement zones coming in close proximity. To resolve these anomolies a special pillow layer is inserted surrounding the uniformly refined elements. We note that

this condition is avoided in the proposed work as all refinement zones are handled simultaneously. Shared memory parallelism is also addressed in Ebeida's work showing examples with notable efficiency, though distributed parallelism is not tackled.

Although Marechal's [3] initial approach is similar, he uses the octree subdivision and subsequent four-column groupings as a dual representation over which a primal set of hexes is later generated. Several impressive examples are shown using the Hexotic software tool using his approach, though initial refined element quality is not reported.

We recognize the important contributions made by these previous works, however to our knowledge, the current literature does not address the parallel consistency issues for distributed domains. We note that a successful 2-refinement strategy must require pairing of adjacent layers or columns of hexes which should be applied in a globally consistent manner. In this work we propose a method that consistently applies pairing in a global manner for distributed domains, and delineates a simple set of local templates that can be consistently applied with minimal inter-processor communication.

2-refinement strategies are most often used as a precursor to an overlay grid method. These methods tend to impose severe constraints on the elements at geometric boundaries. As such, an initial refined base grid that maximizes mesh quality is advantageous. Although most current methods provide adequate topology for 2-refinement, the initial mesh quality can be lacking. As a result we attempt to provide a minimum quality mesh exceeding scaled Jacobian 0.3 prior to the boundary capture and smoothing procedures, providing a more robust platform on which to generate the mesh.

3. Refinement Criteria

In order to apply refinement to an existing Cartesian grid it is necessary to identify elements for uniform refinement. Criteria for refinement may include geometric or physics based factors. For purposes of this study we have incorporated a geometry-based refinement criteria. The hexahedral mesh generation procedure utilized for this study uses a voxel-based representation of the geometry, where volume fractions are defined for each cell of the Cartesian grid which can represent the geometry to the accuracy of the cell resolution. Material interfaces are constructed from the volume fraction data using the primal contouring method described in more detail in [5].

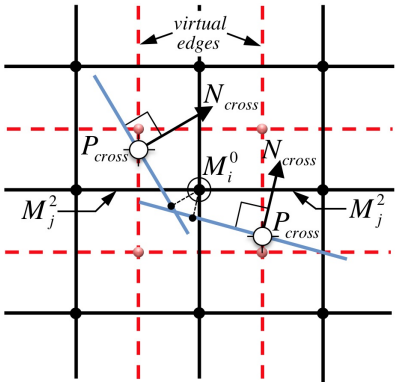


Figure 6: Variables used to move node M_i^0 to an interpolated iso-surface

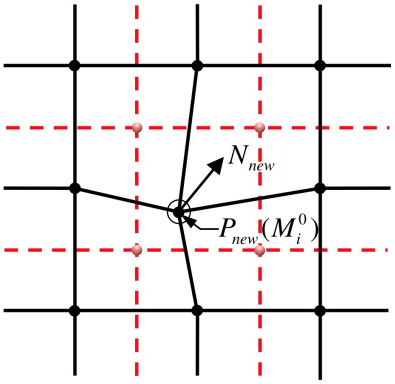


Figure 7: Node M_i^0 after having been moved to the iso-surface

In this procedure, cells are classified according to their dominant material and stair-step interfaces described by the faces between the materials are identified. The nodes of the Cartesian grid on the material interfaces are then adjusted to best approximate the geometry. Figure 6 shows grid node M_i^0 on a material interface that is to be adjusted. To accomplish this, edge crossings shown as P_{cross} are computed on virtual edges connecting cell centers where the material interface is expected to cross a virtual edge. Normals at cell centers can also be computed using a least squares gradient of the material volume fraction. By interpolating between cell centers, a normal N_{cross} can be approximated

for each P_{cross}. The combination of P_{cross} and N_{cross} define a plane which is an approximation of the surface tangent at P_{cross}. The adjusted location of M_i^0 can now be computed as a linear combination of the location of M_i^0, (P_0), projected to each of the tangent planes defined at its neighboring virtual edges as:

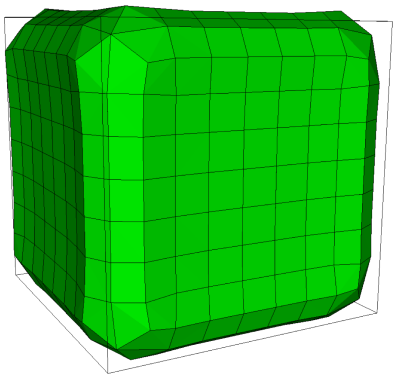
$$P_{new} = \frac{1}{nc} \sum_{i=0}^{i<nc} P_0 - (N_{cross})_i \cdot (P_0 - (P_{cross})_i) \times (N_{cross})_i \tag{1}$$

where P_{new} is the new location of M_i^0 approximating the material interface and nc is the number of edge crossings or tangent planes that have been identified. Figure 7 shows the resulting adjusted location of M_i^0.

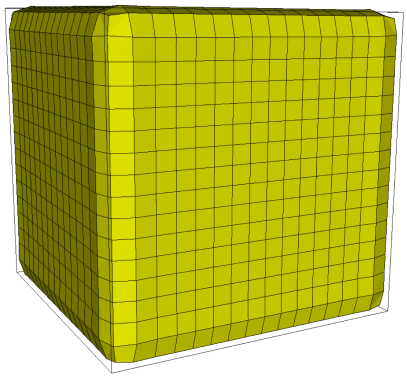
Once the interfaces have been approximated, pillow layers are inserted on either side of the material interfaces and smoothing applied to the internal grid nodes to improve mesh quality.

This procedure has proven effective in capturing a reasonable representation of the surfaces, however the accuracy of the approximation is limited by the local resolution of the grid. While decreasing the constant cell size of the grid is effective in more accurately resolving features of the geometry, it can result in far too many elements in regions where they are not needed. Adaptive refinement of the Cartesian grid is needed in order that element sizes can vary based on geometric features.

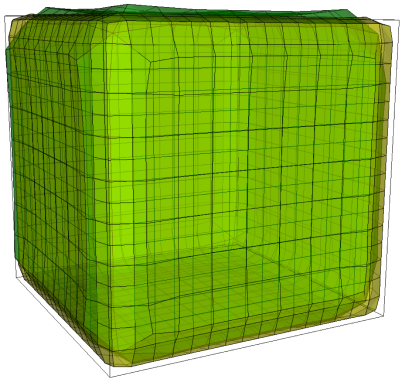
To determine whether a cell should be identified for refinement we can utilize the same procedure represented in equation 1, except on a grid resolution that is one half the size. Consider the two surface representations illustrated in figures 3(a) and 3(b). The surfaces are those generated using equation 1 at level n and $n+1$ respectively from volume fraction data of a simple brick model where level $n+1$ has $8X$ the number of Cartesian cells than level n. It is clear than level $n+1$ will provide a more accurate representation of the brick shape, however some locations on the two surface approximations will be almost identical and do not require refinement. Figure 3(c) shows the surfaces at level n and $n+1$ overlaid, and figure 3(d) shows a close-up at one corner of the brick. We define the



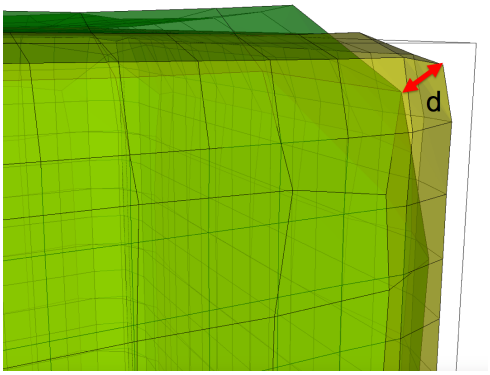
(a) Surface approximation from volume fraction data of a brick shape at level n uniform refinement.



(b) Surface approximation of same volume fraction data at uniform refinement level $n+1$.



(c) Surfaces in (a) and (b) overlaid on top of each other.



(d) Close-up of (c) illustrating Hausdorff distance d between level n and $n+1$.

Figure 8: Example of Hausdorff distance, d, between two surfaces used as criteria for identifying cells to be refined.

Hausdorff distance d between the child and parent surfaces for any point on the child surface. If the Hausdorff distance between any point on the child surface and its parent exceed a user defined threshold, then the containing parent cell is identified for refinement. This criteria is applied recursively where cells of a parent grid at refinement level n are identified for refinement based on the

Hausdorff distance computed between the surface defined at level n and $n+1$.

4. 2-refinement

Algorithm 1 outlines the procedure for applying 2-refinement to a set of selected elements using the templates shown in figures 4 and 5. A more detailed discussion of each of the steps of the algorithm follows using the 2D example illustrated in figure 9 and the 3D example illustrated in figure 10. See the section in parenthesis for additional details on each step.

Algorithm 1 Two Refinement Algorithm

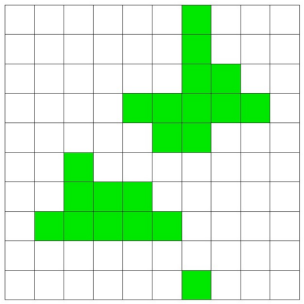
1: Enumerate Cartesian grid (4.1)
2: Identify and mark elements to be uniformly refined (4.2)
3: **for** each direction I, J, K **do**
4: Mark nodes between even-odd layers at marked elements (4.3, 4.6)
5: Apply refinement templates to elements with marked nodes (4.4, 4.7)
6: Expand marked elements to include recently refined elements (4.5, 4.8)
7: **end for**

4.1. Enumerate Cartesian grid

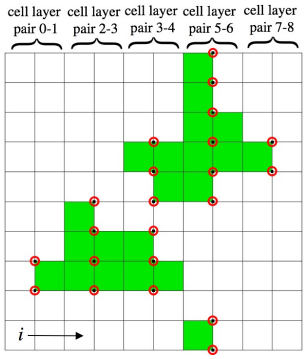
We limit our application to a Cartesian grid that has an arbitrary number of intervals in directions I, J and K. For our purpose [5] the Cartesian grid may be distributed amongst many processors. Figure 11 shows an example distributed Cartesian grid where the domain is shared amongst six processors. Each processor is responsible for independently refining selected elements within its domain. To ensure consistency between processors, we first identify the starting I, J, K indices for each processor based on the global decomposition. Since the 2-refinement procedure relies on grouping pairs of adjacent sheets of hexes, we determine whether the starting indices on each processor are either even or odd. With this information we can consistently provide layer pairs that combine an even-odd layer across the global domain.

4.2. Identify elements to be uniformly refined

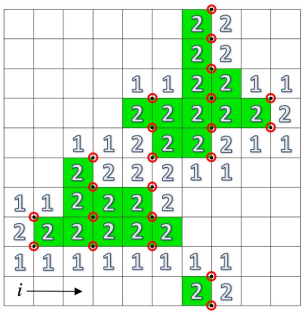
The geometric criteria described in section 3 for identifying cells for refinement is used in this work. The 2D example illustrated in figure 9(a), shaded



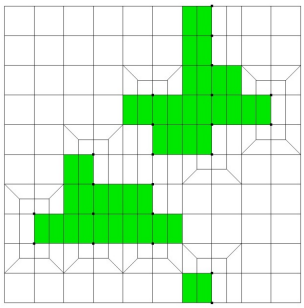
(a) Example 2D Cartesian grid showing elements identified for uniform refinement.



(b) Each even-odd column pair of quads are traversed and nodes between the columns adjacent to at least one marked quad is also marked.



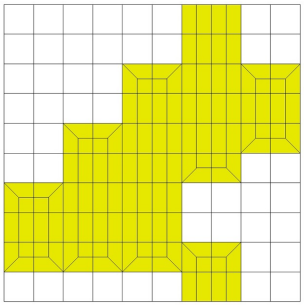
(c) Refinement templates identified based on marked nodes. In 2D only 1 and 2 templates are permitted.



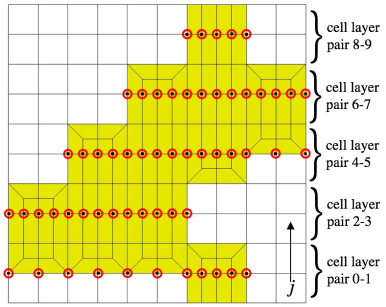
(d) Templates shown in (c) applied to marked nodes in I direction.

cells are those identified for refinement. These cells are guaranteed to contain uniform refinement, or in this case, each cell will be split into four. Although by necessity, additional surrounding quads may also be uniformly refined, transition templates will be applied to other cells to achieve appropriate transitions to maintain a conformal mesh.

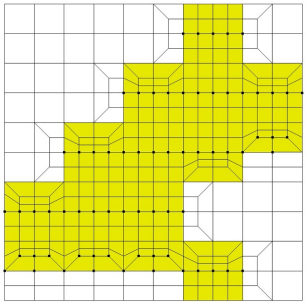
For the 3D case, figure 10(a) shows a simple case of a single element identified for refinement on the interior of the Cartesian grid. This element is guaranteed to be uniformly refined, which for this case, will be split into eight elements.



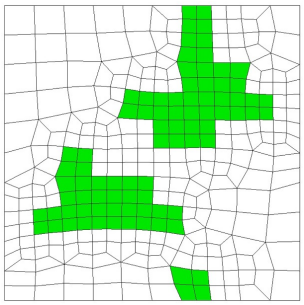
(e) Marked elements now expanded to include refined elements from I direction.



(f) Nodes marked in J direction at every other row adjacent marked elements.

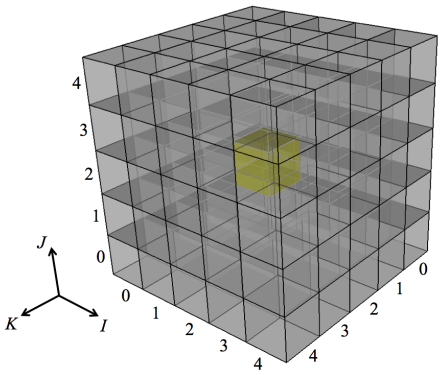


(g) Templates applied on marked nodes in J direction.

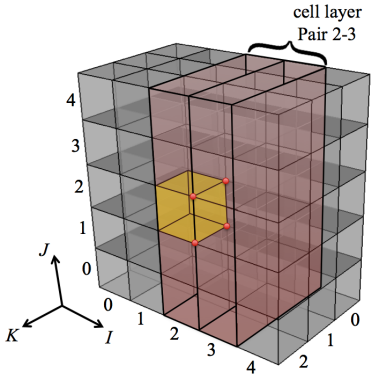


(h) 2D example from (g) following smoothing. Shaded quads illustrate the initial elements identified for uniform refinement.

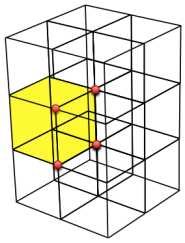
Figure 9: Example procedure used for applying 2-refinement transitions to a set of elements (shown in green) that have been identified for uniform refinement.



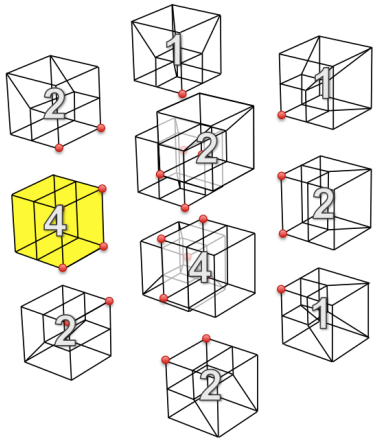
(a) Example 3D Cartesian grid showing a single interior element marked for uniform refinement.



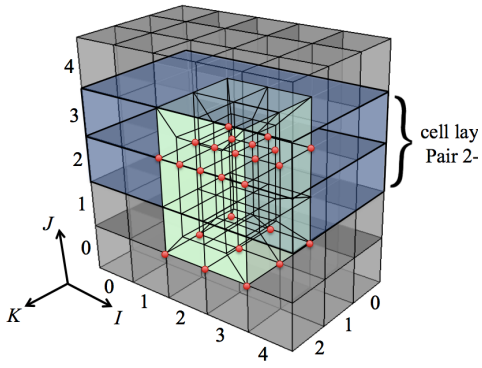
(b) Cut-away view of Cartesian grid. A layer pair is illustrated in the I direction. Grid nodes on the uniform hex between the two layers are marked for refinement.



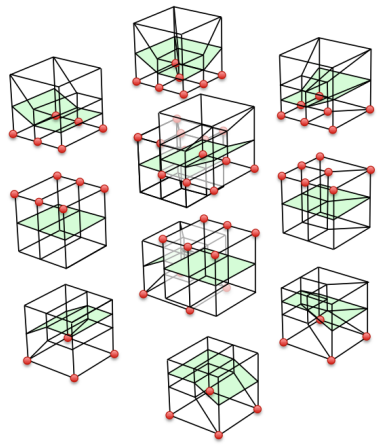
(c) View of the elements immediately adjacent to marked nodes



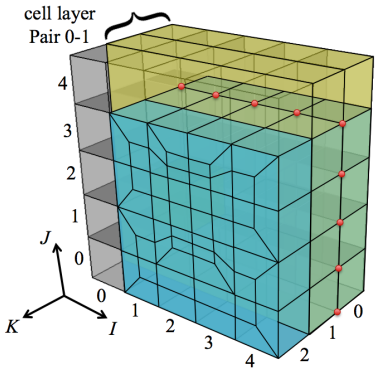
(d) Exploded view of the elements in (c) showing the templates used for refinement in the I direction for each element.



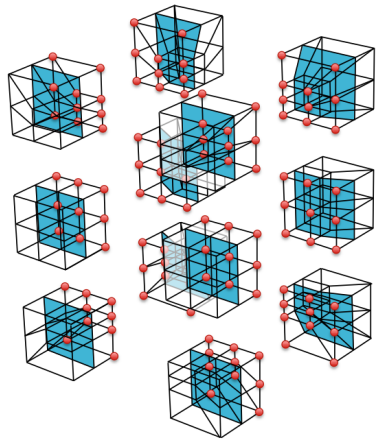
(e) Marked elements now expanded to include refined elements from I direction. An example of a cell layer pair for the J direction is shown with the resulting marked nodes.



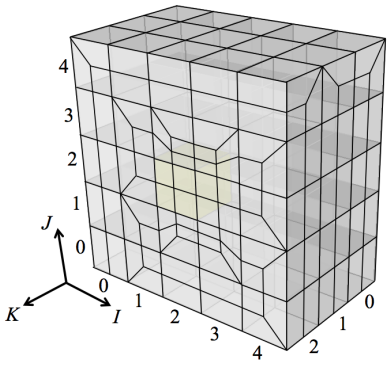
(f) Exploded view of same elements shown in (c) that have been refined in the J direction. Note that 4-templates are used on elements that have been previously refined.



(g) Marked elements now expanded to include all refined elements. An example cell layer pair is illustrated in the K direction with its resulting marked nodes.



(h) Exploded view of elements in (c) where cells have been refined in the K direction



(i) Cut-away view of final 3D refined mesh in all three directions.

Figure 10: Example 3D procedure used for applying 2-refinement transitions to a single interior element that has been identified for uniform refinement.

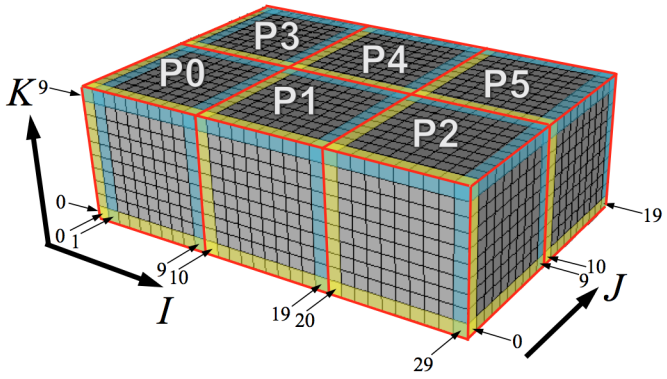


Figure 11: Example distributed Cartesian grid on six processors showing the indices of I, J, K element layers for each processor

4.3. Mark nodes in I direction

To classify elements based on a limited set of templates, we mark some of the nodes in the grid. To enforce the paired layer objective needed for 2-refinement we mark nodes only on every other column of the grid, beginning with the I direction. Each even-odd column pair of cells are traversed and nodes between the columns that are adjacent to at least one uniform refinement cell are marked. This is illustrated on our 2D example in figure 9(b).

Similarly in 3D, we traverse parallel even-odd layers of cells in the I direction. An example layer pair is shown in 10(b) along with the resulting marked nodes.

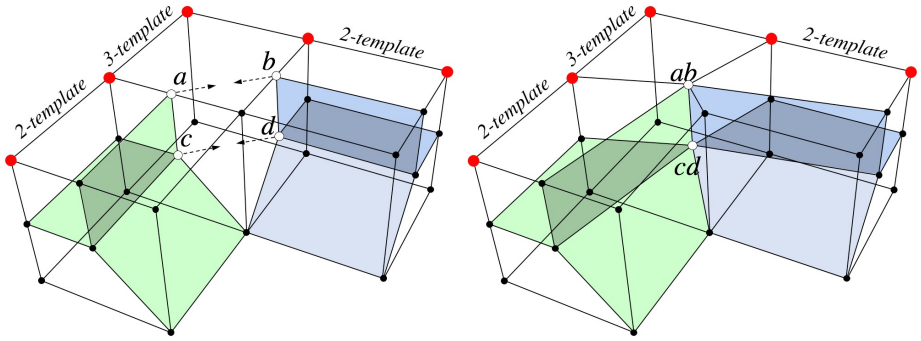
4.4. Apply refinement templates to marked nodes in I direction

Having marked nodes in the I direction, we identify all elements with at least one node marked and apply one of the 2-refinement templates. Figure 4 shows the templates that would be applied for the 2D problem. Figure 9(c) displays either a 1 or 2 based on one of the 2D refinement templates to be used. Figure 9(d) shows the resulting 2D refinement.

For 3D, figure 10(c) show the elements immediately adjacent the marked nodes that will be affected by the refinement. In 3D, the templates shown in figure 5 are used. Figure 10(d) is an exploded view of 10(c) which identifies the templates used and their resulting structure.

It should be noted that the faces of each 3D element shown in figure 5 correspond to one of the 2D templates of figure 4. We also note that this small set of templates shown in figures 4 and 5 is complete, since we limit the marking to every other even-odd set of layers.

Included in the 2-refinement templates is the *3-template*, based on three marked nodes. This template is shown in figure 4(c) for 2D and in 3D in figure 5(c). A similar template is also described by Schneiders in [6]. We note the *3-template* is only used in 3D and is useful for capturing concave regions. Additionally, the 2D *3-template* is used only to describe one of the faces of the 3D *3-template* and would not occur in a planar quad refinement problem. To



(a) Two completed *2-templates* adjacent a partially completed *3-template*. Nodes a and b are merged as well as nodes c and d.

(b) View of completed *3-template* with modified adjacent *2-templates* after merging nodes.

Figure 12: Formation of a *3-template* is accomplished by merging two adjacent *2-templates*.

facilitate its use we merge nodes on two of its adjacent elements. Figure 12 illustrates the *3-template* as it might be used adjacent to two *2-templates*.

4.5. Expand refinement elements from I direction refinement

To prepare for the next stage of refinement we first include all elements that have been previously refined within our set of marked elements. This expands the refinement region, ensuring that further transition templates will only be defined within unrefined elements helping to control element quality and size gradient. A 2D example of this expanded set of marked elements is shown in figure 9(e). Figure 10(e) shows the 3D elements that have been identified for refinement in preparation for refinement in the J direction.

4.6. Mark nodes in J direction

We now repeat a similar procedure to mark nodes in the J direction. We consider all elements grouped within the initial even-odd cell layers of the initial Cartesian grid. All nodes within the refinement region that form the interface between odd and even element layers are marked, including newly added nodes as a result of refinement in the I direction. Figure 9(f) shows a 2D example of

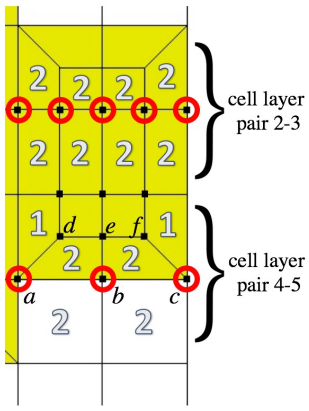
nodes marked in the J direction. The 3D example is shown in figure 9(e) where an example layer pair is illustrated along with the resulting marked nodes.

An exception to the basic procedure of marking nodes between even-odd layers of elements is illustrated in figure 13. Figure 13(a) shows a close-up of some of the even-odd layers of hexes from the example in figure 9(f). Figure 13(b) shows the resulting refinement as it would occur with the basic marking procedure. The *1-templates*, that may result in poorer quality elements, can be eliminated by propagating the marking to the nodes on the interior of the element layer as shown in figure 13(c). For this example, the marking at nodes a, b and c is propagated to nodes d, e and f respectively. Note that nodes a, b and c remain marked in figure 13(c) resulting in quads with all four nodes marked. Quads or hexes with all nodes marked are ignored by the refinement procedure. The resulting 2D quad mesh using the propagation procedure in this example is shown in figure 13(d).

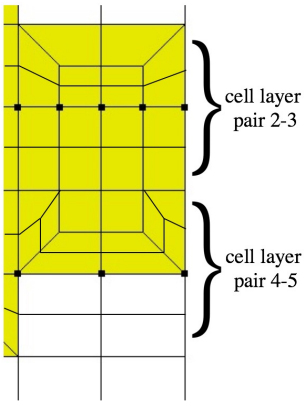
A 3D example of the node marking propagation procedure is also shown in figures 14 and 15. In figure 14 the initial marked nodes shown as nodes q and r in, result in the configuration shown in 14(a). For directions J and K it is common that nodes shown as $a-b-c-d$ will be marked resulting in a *4-template* and a *2-template*. Using the standard template subdivision, this will result in the subdivision shown in (b) which would have poor mesh quality. To avoid this case, we propagate the marking to nodes $e-f-g-h$ shown in (c). Similarly in figure 15 we show a case where a previous *1-template* was refined at node q results in the configuration in 15(a). Where nodes $a-b-c-d$ are marked, a *4-template* and two *2-templates* will result. To avoid the configuration shown in 15(b) we propagate marking from $a-b-c-d$ to nodes $e-f-g-h$ resulting in the configuration shown in (c) with two *4-templates*. We note that an element with all eight nodes marked will remain unrefined.

4.7. Apply refinement templates to marked nodes in J direction

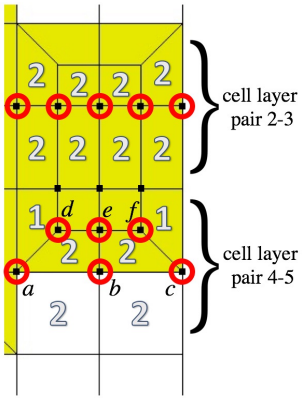
Once the nodes have been marked and appropriately propagated as needed, each element with at least one node marked is refined using the templates il-



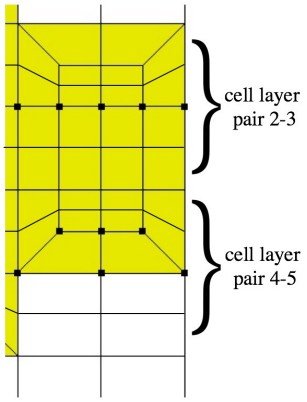
(a) Close-up of node marking and resulting templates based on procedure shown in figure 9(f).



(b) Resulting mesh from basic marking procedure. Note poor angles produced from *1-templates* adjacent the marked nodes a and c in (a).

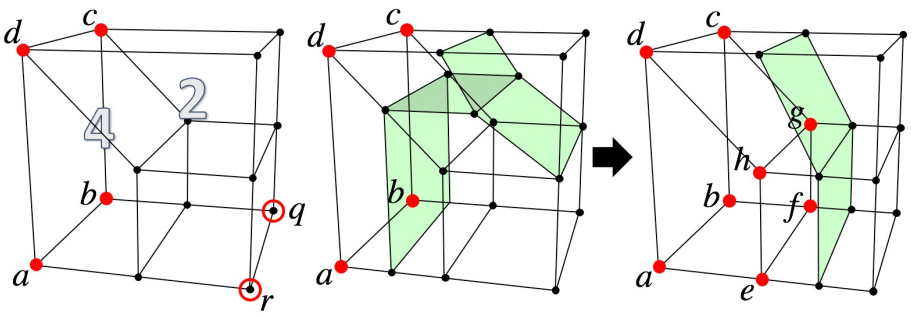


(c) Example of propagation of node marking and resulting templates. The marking at nodes a, b and c are propagated to nodes d, e and f respectively.



(d) Example of resulting mesh where node propagation has been performed.

Figure 13: Example of node propagation in 2D to avoid poor quality elements.

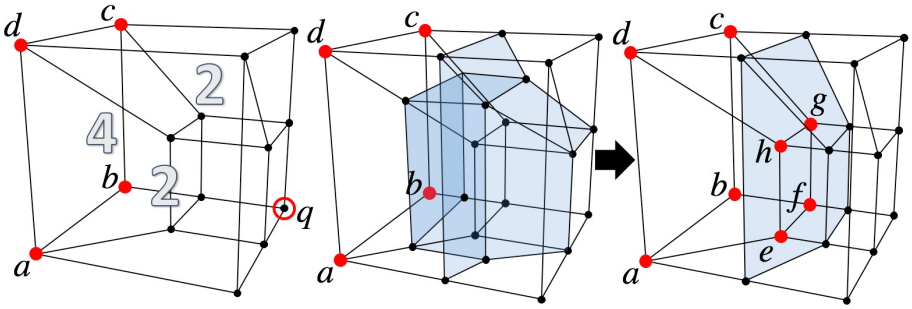


(a) Configuration resulting from a *2-template* where nodes q and r were previously marked.

(b) Resulting refinement when nodes a, b, c, and d are marked. Note poor element where *2-template* is defined.

(c) Resulting refinement when marking has been propagated from nodes a, b, c, and d to e, f, g, and h.

Figure 14: Example of 3D node marking propagation from a *2-template* to improve element quality over standard node marking procedure.



(a) Configuration resulting from a *1-template* where node q was previously marked.

(b) Resulting refinement when nodes a, b, c, and d are marked. Note poor element where *2-templates* are defined.

(c) Resulting refinement when marking has been propagated from nodes a, b, c, and d to e, f, g, and h.

Figure 15: Example of 3D node marking propagation from a *1-template* to improve element quality over standard node marking procedure.

lustrated in 4 and 5. At this stage, our 2D refinement example is complete. To illustrate its effectiveness, figure 9(h) shows our 2D example after smoothing has been performed. The shaded elements show the initial region that was marked for uniform refinement.

4.8. K direction refinement

For the 3D problem we continue the procedure by expanding the set of marked elements to all elements that have been previously refined by both I and J direction refinement procedures. Even-odd pairs of elements in the K direction are then used in a similar manner to mark nodes between the layers, including nodes from elements previously refined. The procedure for propagating node marking to the interior of the elements is also used for the K direction. Finally, all elements with at least one node marked are then refined using the templates illustrated in figure 5.

4.9. Additional node marking requirements in 3D

We also note two other conditions in 3D where it is necessary to mark additional nodes that may not be included in the set of marked elements. These cases are illustrated in figures 16 and 17. If we consider the plane between the even-odd layers of elements, it is possible to mark nodes such that invalid templates could be identified. The case illustrated in figure 16 shows face A at the even-odd layer interface where only nodes a and c would be marked. Since diagonal marking on a single face is not handled by our templates, we simply add the other diagonal nodes b and d into the set of marked nodes.

Figure 17 illustrates a case where faces A and B lie between the even-odd layers of hexes with surrounding marked elements as shown. Without additional node marking, both faces A and B would require *3-templates* for refinement. Because a *3-template* will merge nodes from neighboring elements, we restrict the use of the *3-template* so that two adjacent *3-templates* cannot be defined. When a case such as this is identified, the fourth node, in this example node f, is marked eliminating this condition.

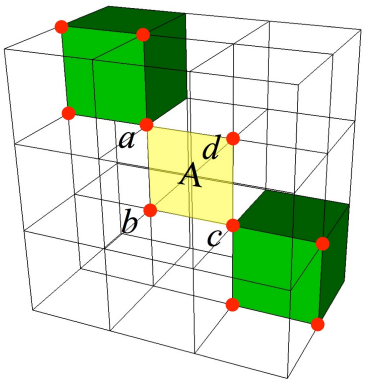


Figure 16: Nodes b and d at face A would also be marked to avoid an invalid template for adjacent elements at A

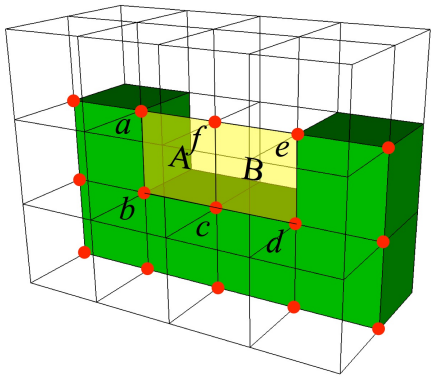


Figure 17: Node f would also be marked to avoid two adjacent *3-templates*

5. Multiple levels of refinement

In the previous section we outlined a strategy for two refinement involving transitioning from 1 to 8 elements. It is also possible to incorporate multiple levels of refinement where a second level would split 8 elements into 64, a third into 256 and so on. Because the proposed 2-refinement strategy will work only within a structured set of elements, we limit the introduction of additional levels of refinement to only uniformly refined parent elements. We also note that since our strategy can take up to three additional hex layers to effect the transition, we first expand the uniform refinement zone of the parent elements to allow for at least three layers surrounding the child uniform refinement. This is illustrated in figures 18 and 19. Once the lowest level of refinement is determined based on local feature sizes, a balancing procedure is performed to ensure sufficient transition layers are provided for each level of refinement. In figure 18 we use two additional layers of the initial hexes (shown in green) surrounding the second level of refinement (shown in red). After the first level of refinement is complete, four layers of uniformly refined hexes are available to be used for the transition elements of the second level of refinement shown in figure 19. In figure 20 we show the same mesh following a smoothing operation.

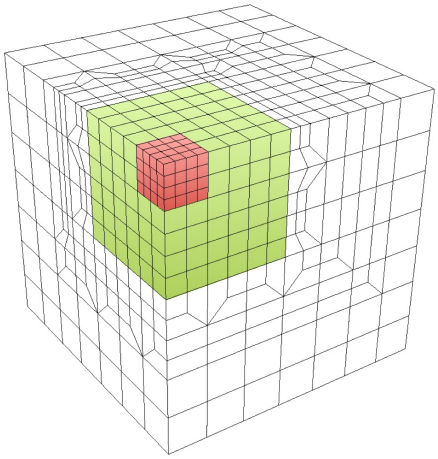


Figure 18: Example 3D refinement showing first level of refinement from green elements. Green elements will serve as transition zone for second level of refinement from uniformly refined red elements

To perform the refinement at subsequent levels, the same procedure outlined in Algorithm 1 is performed. This can be accomplished recursively for as many levels as desired. Since the parent elements will have already been uniformly refined, the parent element layers will each contain an even-odd set of child layers that can be used for the marking procedure, thus maintaining the parallel consistent nature of the algorithm through multiple levels of refinement.

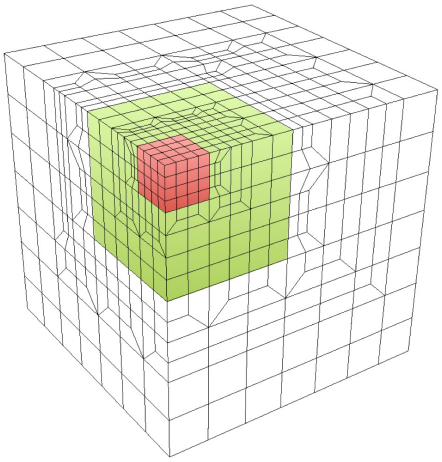


Figure 19: Transition elements for second level of refinement added.

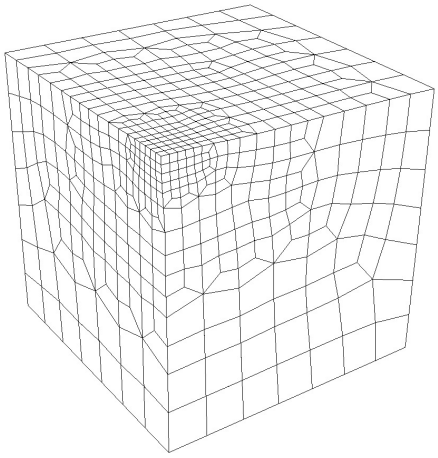


Figure 20: Smoothing applied to the model in figure 19

6. Parallel implementation

One of the principal objectives of this work is to provide a scalable implementation that can be used in a massively parallel distributed environment. To accomplish this it was necessary to provide a parallel consistent algorithm that required minimal inter-processor communication. For our purposes we limit the global domain to be a Cartesian grid that has been distributed across multiple processors, where each domain is itself a Cartesian grid. Section 4.1 describes the initial global enumeration of the grid where the starting i, j, k indices for each processor are identified as either even or odd. This provides the basis for the even-odd layer pairs used in the node marking procedure and ensures that all templates will be conformal across processors with minimal additional communication.

In order to maintain a conformal mesh across processors, two required instances of inter-processor communication are noted. Communication is effected by using ghosted elements at the boundaries of each domain. A four processor example is illustrated in figures 21 and 22. Initially, two layers of elements are included in the ghosted regions of each processor. For each hex and its refined children, we keep track of their owning processor, where any refined element is owned by its parent's processor. Redundant and identical refinement operations are performed on the owning processor and its ghosts on neighboring processors. With consistent node marking provided by the global even-odd layer pairing, we are guaranteed a conformal mesh across processor domains provided the following two communication steps are effected.

6.1. Communication of node marks

Although the basic node marking procedure described in section 4 can be done independently on each processor, the additional cases for node marking described in section 4.9 may expand the marking into a neighboring processor. In addition, when multiple levels of refinement are used, as described in section 5, the expansion of the uniformly refined region necessary to include multiple

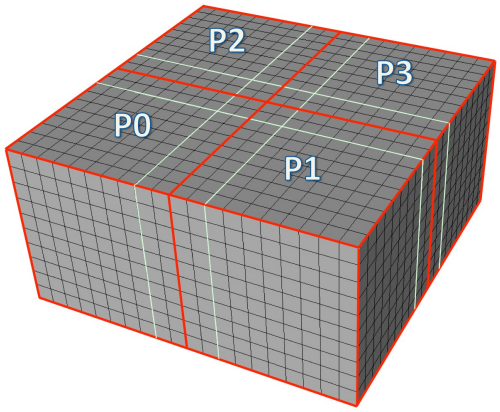


Figure 21: Example parallel distribution of Cartesian grid on four processors

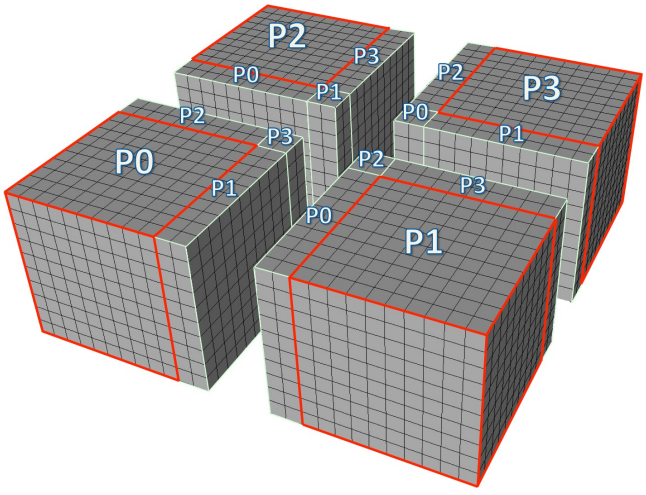


Figure 22: Exploded view of 4 processors where ghosted elements of neighboring processors are included for each processor

levels may extend across processors. To allow for these cases, nodes marked within the ghosted region of a neighboring processor are communicated prior to performing template-based refinement. This is done by identifying nodes on the current processor that have been marked that have a corresponding ghosted node on a neighboring processor. Messages are passed between neighboring processors to update the marked state to reflect the status of each node's owning processor. Communication of marked nodes is done once for each direction I, J and K. In algorithm 1 above, this communication would occur following line 4, prior to performing refinement in each direction.

6.2. Communication of 3-template *node movement*

For most templates, Node locations can be simply determined as the mid-point of edges, faces and hexes. This provides a parallel consistent implementation across processor boundaries. The one exception to this case is when a *3-template* is applied near a processor boundary. Figure 23 shows a 2D representation where a *3-template* is identified on processor 0 and node n_0 is relocated to form the template. Since processor 1 has only a single node from the *3-template*, it will define a different location for the same node (n_1) at the mid-point of an edge. To account for this case, we include an additional communication step following template refinement to update node locations. This is done by keeping track of node locations updated as a result of *3-template* operations on the owning processor. These new node locations are communicated to neighbor processors and locations updated if the node lies at a domain boundary. In algorithm 1 above, this communication would occur following line 5, following refinement in each direction.

6.3. Iteration of node marking communication

While these two communication operations will generally suffice, it is possible that the node marking procedure in the ghost cells of one of the processors will result in the cases described in section 4.9 and figures 16 and 17 resulting in a condition where additional nodes must be marked. When new nodes have

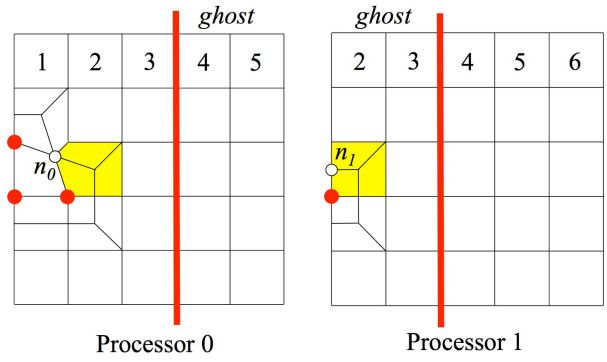


Figure 23: Illustration of discrepancy between node location n_0 and n_1 on neighboring processors resulting from a *3-template* near a processor boundary.

been marked, communication to provide the new marking data to neighbors must be done. To facilitate these cases, following node communication, we find any new cases requiring additional node marking and communicate the result to neighboring processors. This procedure iterates until no additional node marking is necessary.

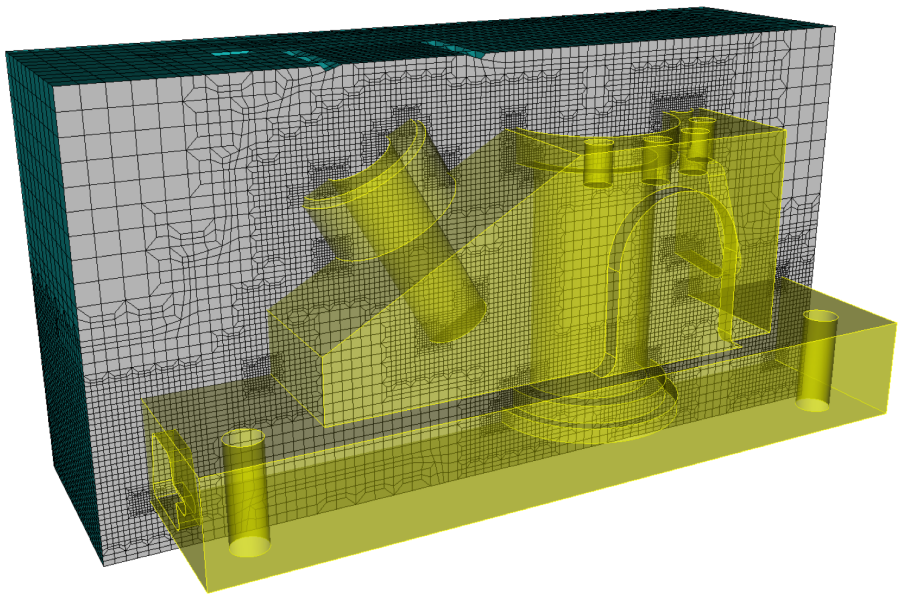
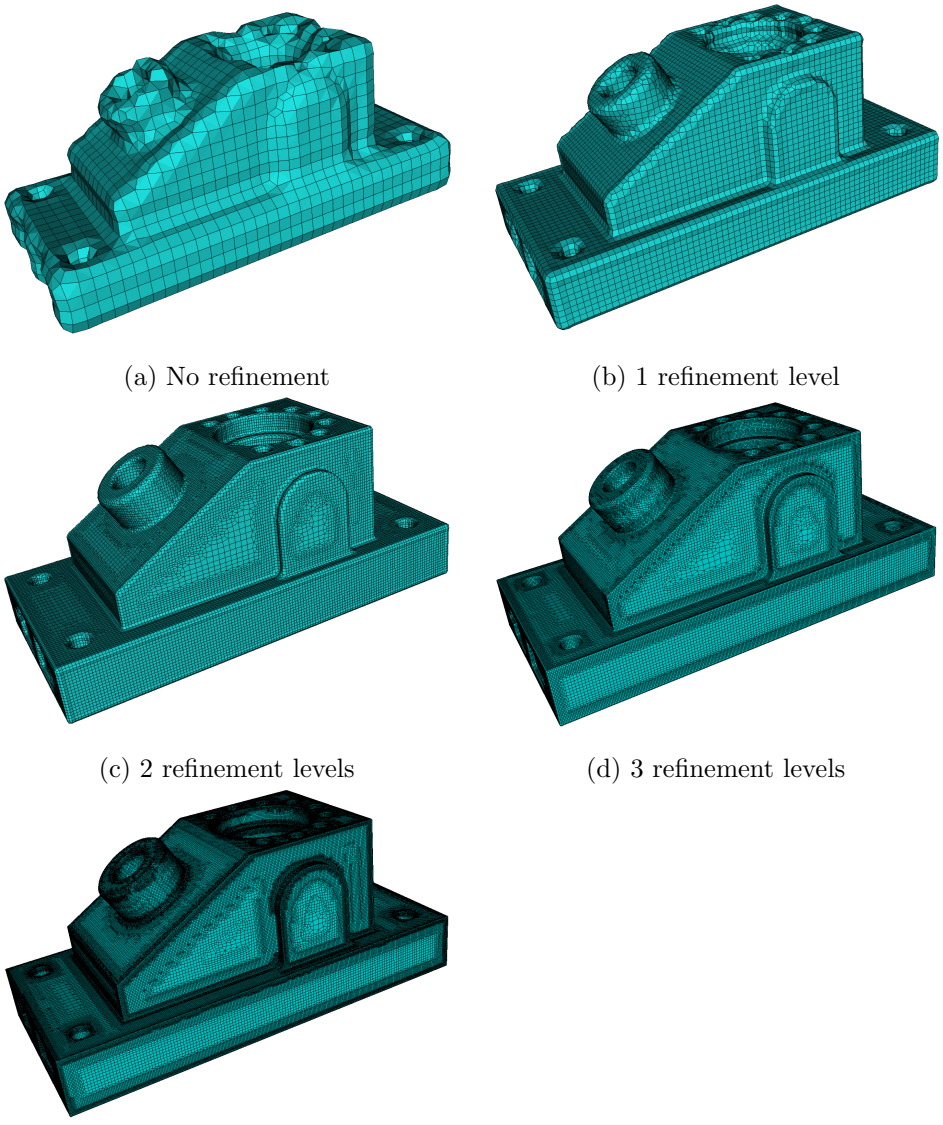


Figure 24: Cut-away of refined base mesh for anc101 model at 4 levels of refinement.

7. Examples

We illustrate the proposed algorithm with a limited set of examples. The intended objective of the proposed refinement algorithm is its use within Sandia's Sculpt [5] tool. In this tool, a regular, user-defined Cartesian grid is specified and the refinement procedure applied based on the geometric criteria described in section 3. Once a refined grid is complete, the Sculpt procedure, as outlined in [5] is employed to generate an all-hex finite element mesh.

We show examples of the refined Cartesian grids in figures 24, 27 and 31 where we have applied up to 4 levels of refinement. We note that following refinement, the minimum scaled Jacobian for each refined grid is 0.329. Example results using the Sculpt tool are also shown at varying resolutions in figures 25 and 26 for the anc101 model, figures 28 and 29 for the Stanford bunny model, and figures 31 and 32 for the Stanford dragon model.



(a) No refinement

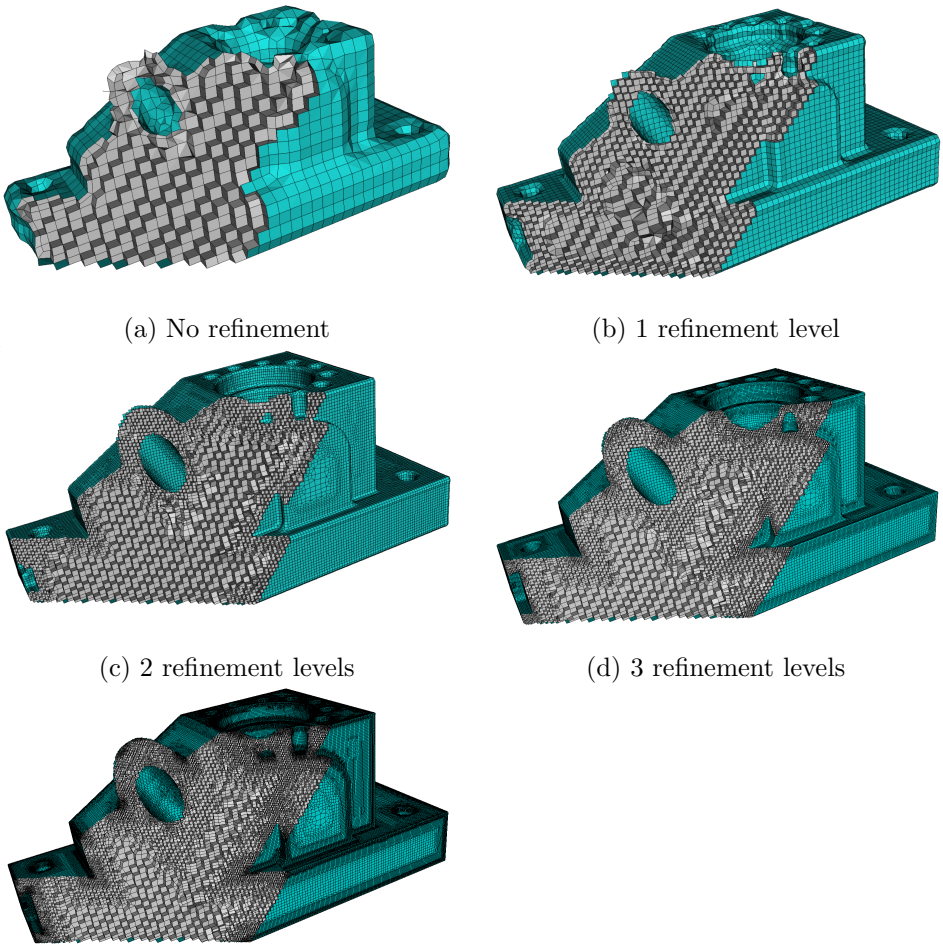
(b) 1 refinement level

(c) 2 refinement levels

(d) 3 refinement levels

(e) 4 refinement levels

Figure 25: anc101 model meshed using adaptive refinement at various resolutions.



(a) No refinement

(b) 1 refinement level

(c) 2 refinement levels

(d) 3 refinement levels

(e) 4 refinement levels

Figure 26: cut-away view of anc101 model meshed using adaptive refinement at various resolutions.

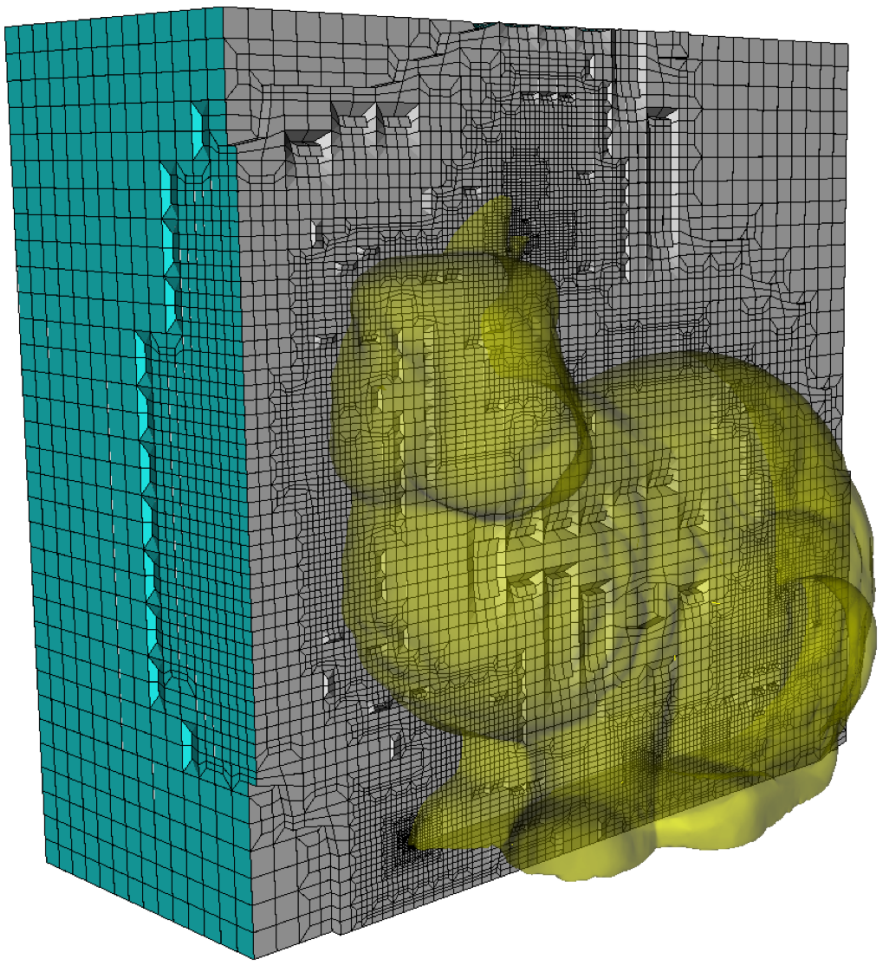
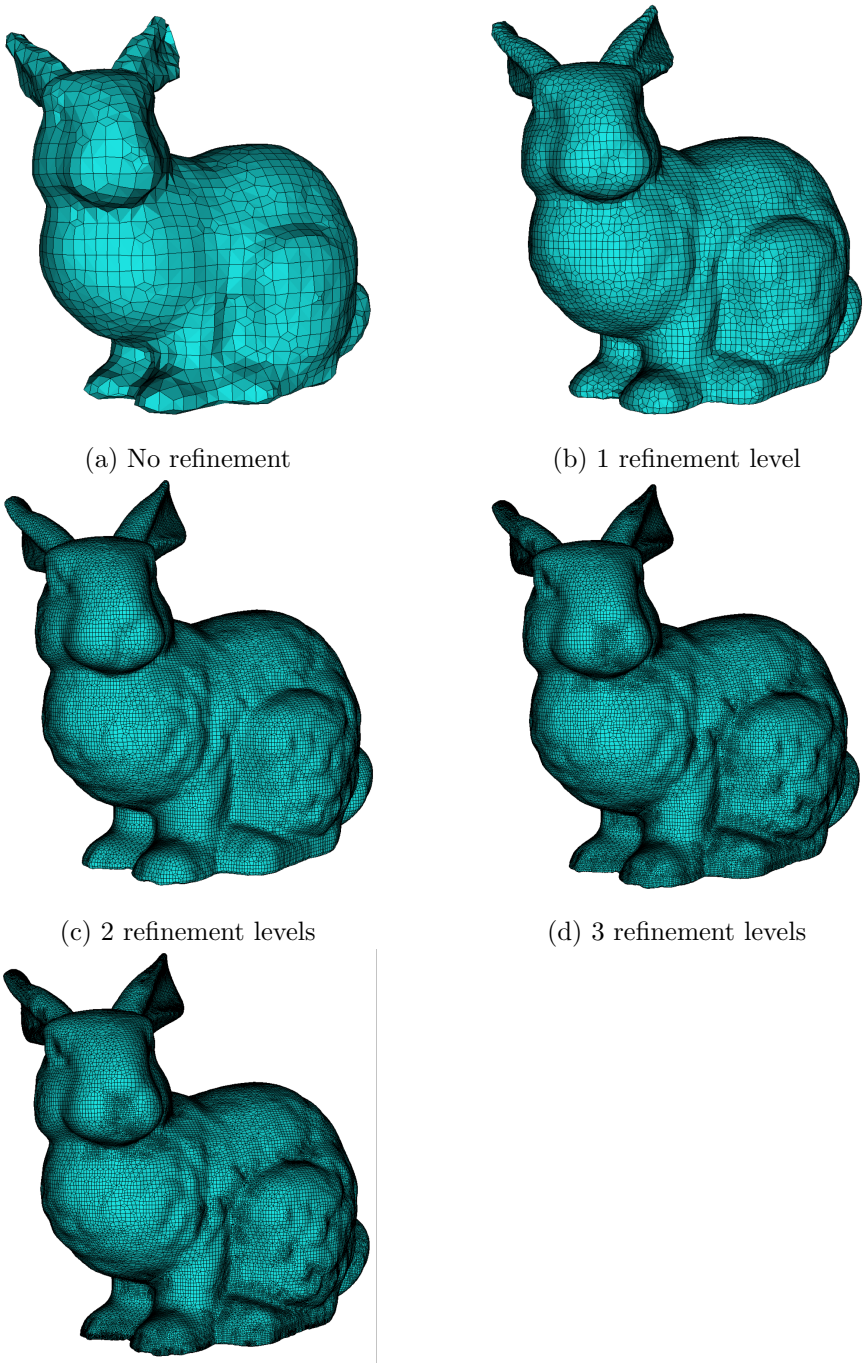


Figure 27: Cut-away of refined base mesh for Stanford bunny model at 4 levels of refinement.



(a) No refinement

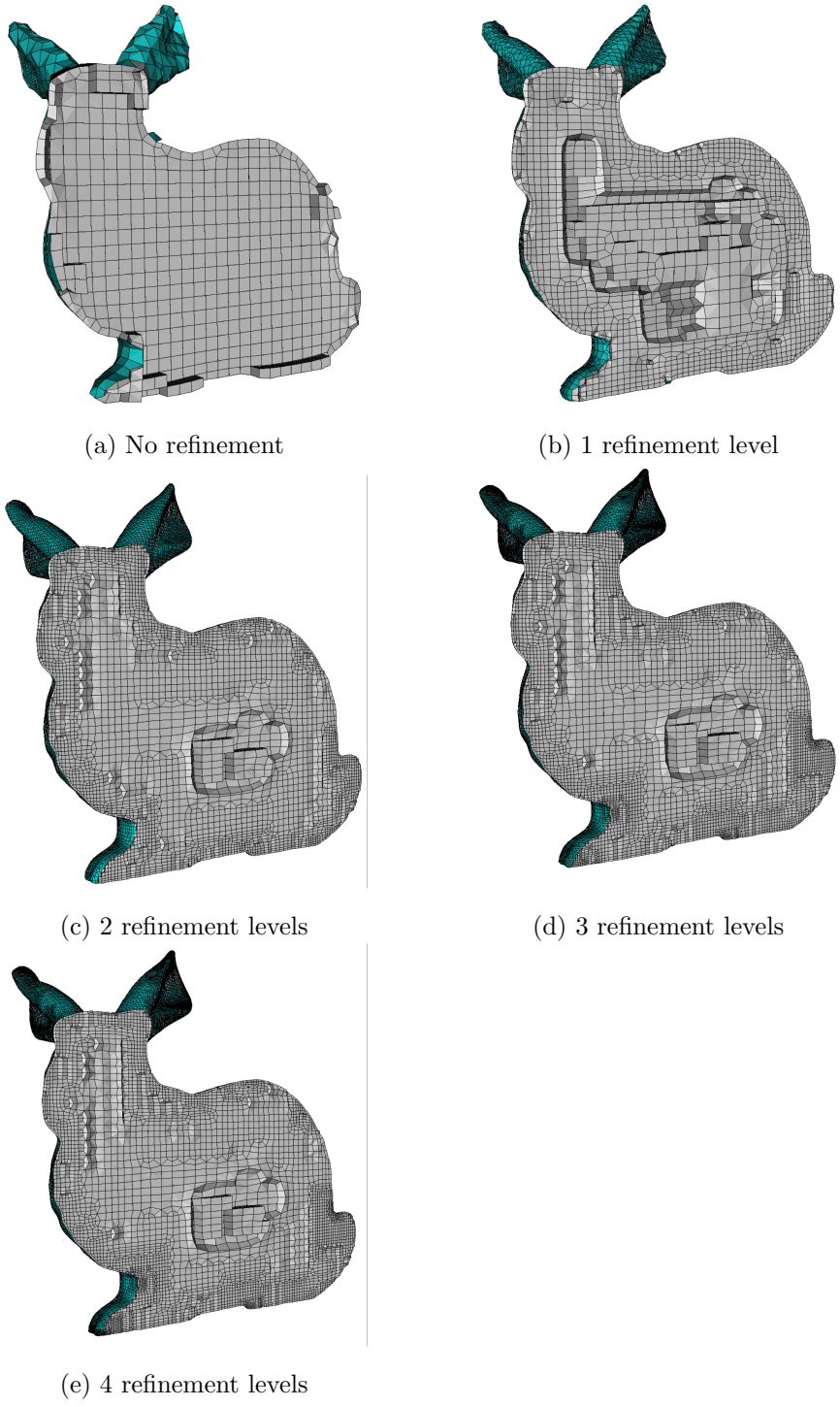
(b) 1 refinement level

(c) 2 refinement levels

(d) 3 refinement levels

(e) 4 refinement levels

Figure 28: Stanford bunny model meshed using adaptive refinement at various resolutions.



(a) No refinement

(b) 1 refinement level

(c) 2 refinement levels

(d) 3 refinement levels

(e) 4 refinement levels

Figure 29: cut-away view of Stanford bunny model meshed using adaptive refinement at various resolutions.

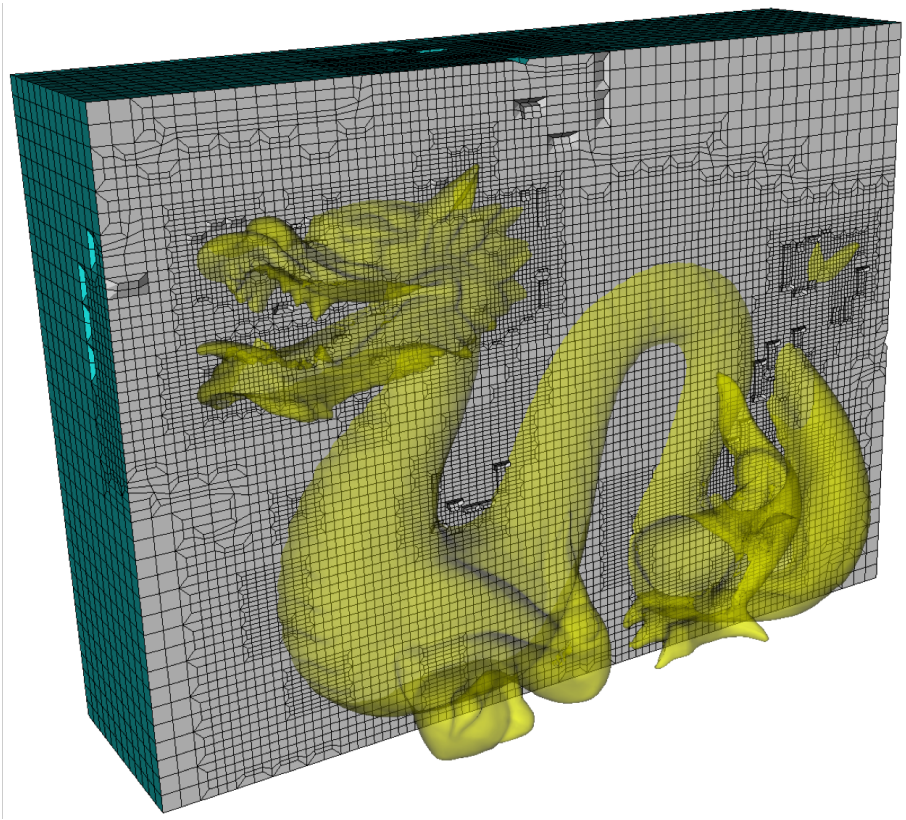
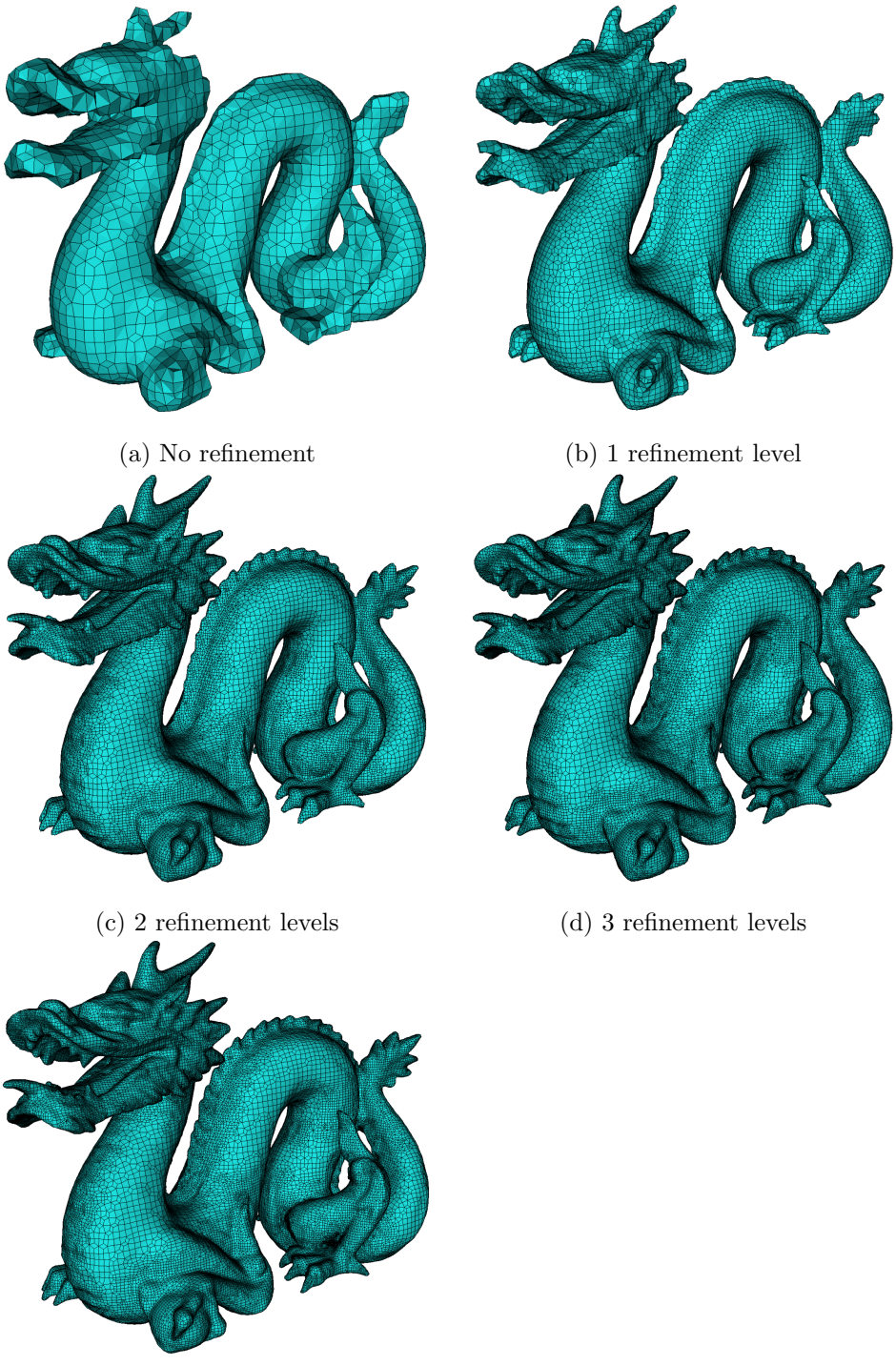


Figure 30: Cut-away of refined base mesh for Stanford dragon model at 3 levels of refinement



(a) No refinement

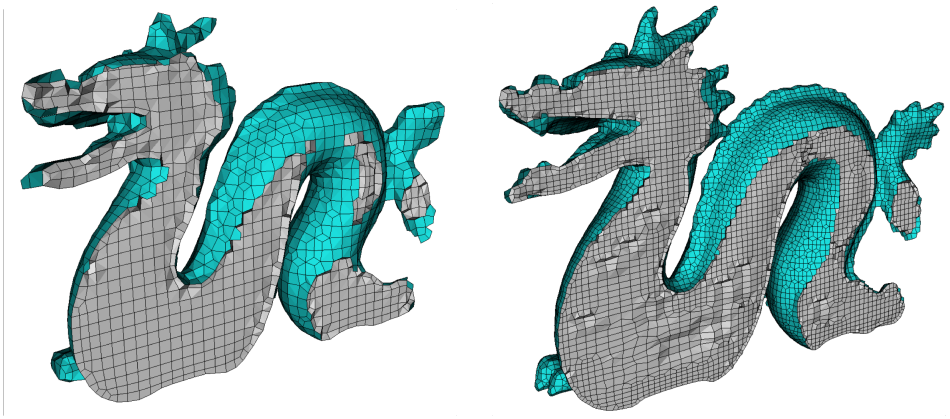
(b) 1 refinement level

(c) 2 refinement levels

(d) 3 refinement levels

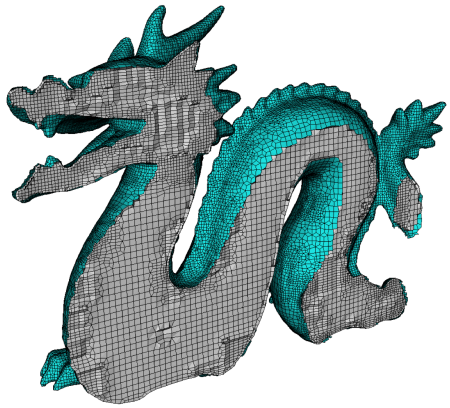
(e) 4 refinement levels

Figure 31: Stanford dragon model meshed using adaptive refinement at various resolutions.

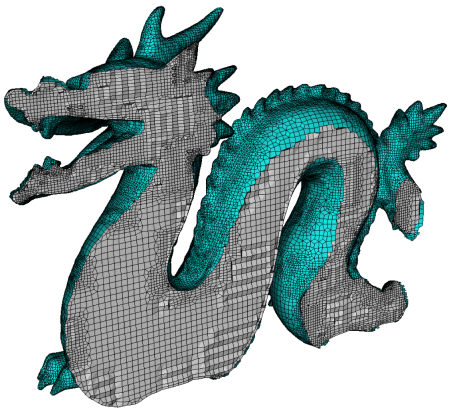


(a) No refinement

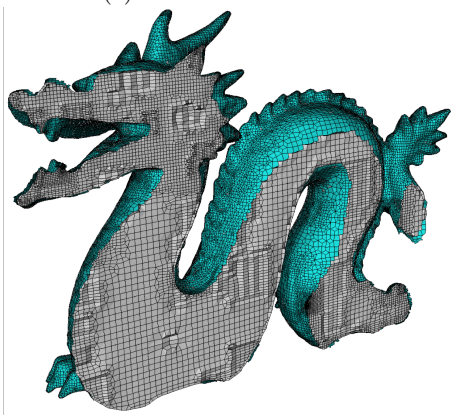
(b) 1 refinement level



(c) 2 refinement levels



(d) 3 refinement levels



(e) 4 refinement levels

Figure 32: cut-away view of Stanford dragon model meshed using adaptive refinement at various resolutions.

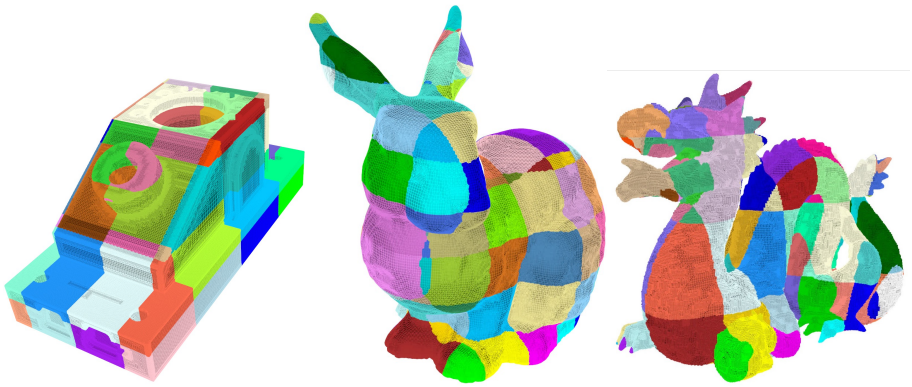
8. Performance

A performance study was performed with results illustrated in figures 34, 33 and table 1. Tests measured the time to refine an initial Cartesian grid bounding each of the models at two different resolutions. The first series of tests began with the resolution of the Cartesian grid at about 30-40 thousand cells and apply up to four levels of refinement. The final meshes for these cases are illustrated in figures 24 to 32. The second series, labelled 4X, began with intervals one fourth the size, or 64 times the initial resolution. A maximum four levels of refinement was also applied to these. The numbers of initial Cartesian cells and final number of cells for each case are shown in table 1.

We were able to observe that for each individual series of test cases, the exact same number of refined cells were produced regardless of the number of processors used. We also noted that minimum scaled Jacobian before performing smoothing for all cases was approximately 0.329.

Observations of the data in figure 34 showed that the initial test cases are reasonably efficient but deviate substantially from the ideal for higher numbers of processors. For example, the ideal scaling line is shown for the anc101 test case in figure34. Performance from one to two processors drops from 1450 seconds to about 800 seconds. However from 512 to 1024 processors it drops only from 84 seconds to 64 seconds. We note that the spatial decomposition required for 1024 processors in this case results in a distribution of only about 30 elements per processor. Since two layers of ghost cells are required for the algorithm, maintaining and updating ghost cells becomes the bulk of the computation. We note however for the 4X models, their performance graphs tend closer to the slope of the ideal since maintaining ghosted cells are a proportionally smaller part of the domain.

We also noted that memory can become an issue for larger models when run on smaller numbers of processors. Indeed we were unable to run the 4X models with less than 32 processors. Conversely we were unable to run the smaller models on higher numbers of processors since the distribution of initial cells



(a) anc101 64 processsors.

(b) bunny 256 processsors.

(a) dragon 128 processsors.

Figure 33: Example models used in performance study shown in figure 34 and table 1. Resulting internal cells for refined Cartesian grid prior to running Sculpt [5] are displayed. Colors represent individual processors.

Table 1: Refinement Scaling Study

	anc101	bunny	dragon	anc101 4X	bunny 4X	dragon 4X
Initial Num. Cells	32,400	38,880	44,100	2,073,600	2,488,320	2,882,400
Final Num. Cells	6,993,400	1,572,306	4,211,206	15,298,784	14,532,960	40,066,240

became too small per processor.

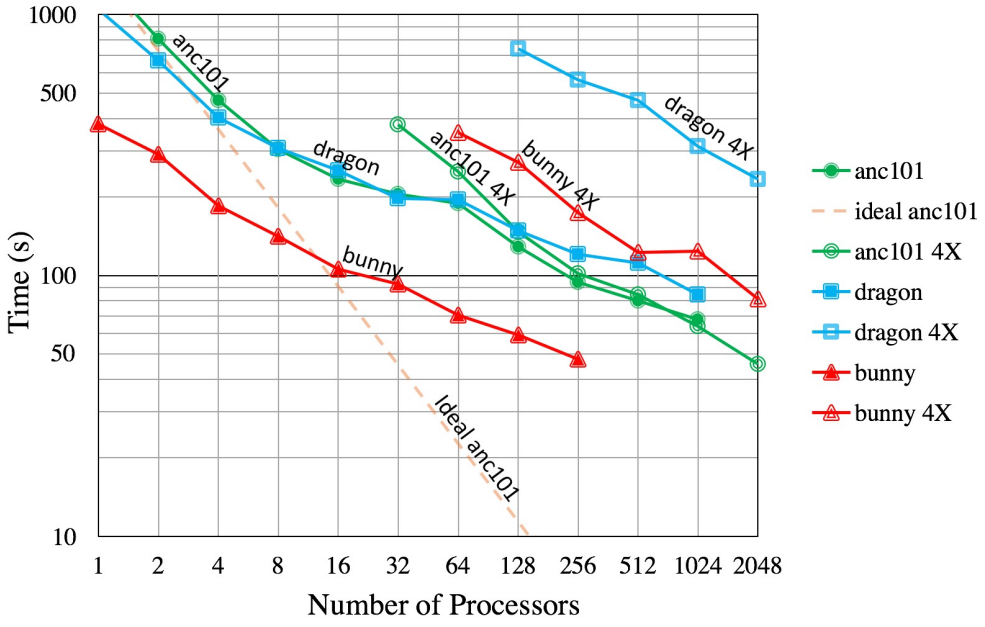


Figure 34: Log plot of performance of refinement vs number of processors for three different models. Maximum 4 levels of refinement are performed. The 4X models use the same geometry but begin at higher resolution as outlined in table 1. All tests were performed on Sandia's TLCC2 Chama platform (1,232/19,712 nodes/cores 2.6GHz Intel, 64GB/node).

9. Conclusion

2-refinement is generally preferred over 3-refinement methods for capturing features in overlay grid methods. We have proposed and demonstrated a 2-refinement procedure that is applicable for a distributed memory parallel environment that maintains consistent hex layer pairing across processors. A modest set of refinement templates combined with a node marking procedure result a relatively simple implementation that avoids some of the layer pairing issues arising in other published methods. We note that this method avoids the non-manifold resolution and additional pillowing operations sometimes required by other 2-refinement procedures [11][12]. It also extends previous layer pairing procedures [8][10] to effectively incorporate concave regions while using a simple set of refinement templates. Examples show that mesh quality exceeds 0.3 scaled Jacobian in all cases prior to any smoothing, which appears to be a significant improvement over alternative methods. The method also provides for multiple levels of refinement.

In contrast to 3-refinement and other published 2-refinement methods, we recognize that in order to achieve the observed mesh quality, the transition zone may extend up to three additional layers from the uniform refinement zone. The additional space provided by the proposed method provides for smooth size gradients in transition regions which can dramatically improve mesh quality in the final overlay grid mesh.

In this work we have presented an algorithm for 2-refinement along with results to demonstrate its effectiveness. Future work will further optimize this method with the sculpt software [5] running in a massively parallel environment. While initial results have shown effective results on up to 256 processors with MPI, future work will require performance enhancements as we examine its usefulness and further scalability on massively parallel HPC machines.

10. Acknowledgements

Funding for this work was provided by the US Department of Energy Advanced Scientific Computing program. Special thanks to Brett Clark and Randy Morris for providing review and feedback. CAD Models used in examples provided by Ansys, Inc.

References

[1] Y. Zhang, C. Bajaj, Adaptive and quality quadrilateral/hexahedral meshing from volumetric data, Computer methods in applied mechanics and engineering 195 (9) (2006) 942–960.

[2] Y. Ito, A. M. Shih, B. K. Soni, Octree-based reasonable-quality hexahedral mesh generation using a new set of refinement templates, International Journal for Numerical Methods in Engineering 77 (13) (2009) 1809–1833.

[3] L. Marechal, Advances in octree-based all-hexahedral mesh generation: Handling sharp features, in: 18th International Meshing Roundtable, 2009, pp. 65–84.

[4] R. Schneiders, Handbook of Grid Generation, no. 21, CRC Press LLC, 1999, Ch. Quadrilateral and Hexahedral Meshes.

[5] S. J. Owen, M. L. Staten, M. C. Sorensen, Parallel hexahedral meshing from volume fractions, Eng. with Comput. 30 (3) (2014) 301–313. doi:10.1007/s00366-012-0292-8.
URL http://dx.doi.org/10.1007/s00366-012-0292-8

[6] R. Schneiders, Algorithms for quadrilateral and hexahedral mesh generation, Proceedings of the VKI Lecture Series on Computational Fluid Dynamic, VK-LS 4.

[7] T. J. T. S. A. Mitchell, Pillowing doublets: Refining a mesh to ensure that faces share at most one edge, in: Proceedings 4th International Meshing Roundtable, 1995, pp. 231—240.

[8] S. O. N. Harris N, S. Benzley, Conformal refinement of all-hexahedral element meshes based on multiple twist plane insertion, in: Proceedings 13th International Meshing Roundtable, 2004, pp. 157—168.

[9] S. J. O. J. Edgel, S. E. Benzley, An adaptive grid-based all hexahedral meshing algorithm based on 2-refinement (research notes), in: 19th International Meshing Roundtable, 2010.

[10] J. B. Malone, Two-refinement by pillowing for structured hexahedral meshes, Master's thesis, Brigham Young University (2012).

[11] M. S. Ebeida, A. Patney, J. D. Owens, E. Mestreau, Isotropic conforming refinement of quadrilateral and hexahedral meshes using two-refinement templates, International Journal for Numerical Methods in Engineering 88 (10) (2011) 974–985.

[12] Y. Zhang, X. Liang, G. Xu, Proceedings of the 21st International Meshing Roundtable, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, Ch. A Robust 2-Refinement Algorithm in Octree and Rhombic Dodecahedral Tree Based All-Hexahedral Mesh Generation, pp. 155–172. doi:10.1007/978-3-642-33573-0_10.
URL http://dx.doi.org/10.1007/978-3-642-33573-0_10